

Session Facilitator Guide

"Selenium Automation Framework + GitHub Copilot Custom Agents"

Duration: 90 minutes | Audience: QE / Developer team

Session Flow at a Glance

#	Topic	Time	Cumulative
1	Target Application Walkthrough	5 min	0:05
2	Framework Architecture (README)	10 min	0:15
3	Test Classes Deep Dive	15 min	0:30
4	Configuration & Test Data	5 min	0:35
5	Page Objects & Driver	15 min	0:50
6	TestNG Suites	5 min	0:55
7	Live Test Run + Reports	10 min	1:05
8	GitHub Copilot Custom Agent	20 min	1:25
9	Q&A Buffer	5 min	1:30

PART 1 — The Target Application (5 min)

Show: <https://www.franchising.inspirebrands.com/>

Talking points:

- This is a Squarespace-hosted CMS with 7 franchise brand pages
- Open the "Our Brands" dropdown → walk through all 7 brands (Arby's, Baskin-Robbins, BWW, BWW GO, Dunkin', Jimmy John's, SONIC)
- Navigate to <https://www.franchising.inspirebrands.com/arbys> — this is the page we automate
- Point out: hero section, "Why Arby's?", qualification requirements, "GET STARTED" CTA, footer
- **Key challenge to mention:** Squarespace renders text inside `<span>` elements, uses curly apostrophes (U+2019), and CSS scroll animations — all of which make locators harder than they look

PART 2 — Framework Architecture via README (10 min)

Open: `README.md`

Walk through each section:

Architecture diagram — read the layered diagram top-down:

```
TestNG Suite XML → Test Layer → Page Object Layer → Factory → Infrastructure → Config
```

Project structure tree — explain each folder:

- `src/main/java/com/inspire/` — framework code (not tests)
- `src/test/java/com/inspire/tests/` — test classes
- `src/test/resources/` — config + brand properties
- `testng-suites/` — suite XML files
- `mvnw.cmd` — Maven Wrapper (no system Maven needed)

Design patterns table — briefly name each pattern (will demo in code shortly)

Adding a New Brand — 4 steps — emphasise: *zero existing files modified*

PART 3 — Test Classes Deep Dive (15 min)

3a. `AbstractBrandTest.java` (5 min)

Open: `src/test/java/com/inspire/tests/brands/AbstractBrandTest.java`

Talking points:

- This is an **abstract class** — never runs directly
- Contains TC-B-01 through TC-B-12 — the 12 tests every brand gets **for free**
- `getBrandPage()` is the **Template Method Pattern hook** — subclasses provide the page object
- `@BeforeMethod openBrandPage()` — calls `brandPage.open()` before each test
- Point to `brandPage.getBrandDisplayName()` — never hardcoded brand names
- **Audience question:** "If I add Dunkin' tomorrow, how many test lines do I write to get these 12 tests running?" → Answer: **zero**

3b. `ArbysTest.java` (5 min)

Open: `src/test/java/com/inspire/tests/brands/ArbysTest.java`

Talking points:

- `extends AbstractBrandTest` → inherits all 12 common tests automatically

- `getBrandPage()` returns `new ArbysPage(driver)` — one line to wire the brand
- `arbysPage()` private helper — avoids repeated casts
- TC-A-01 through TC-A-09 — Arby's-specific validations only
- Point to TC-A-05/TC-A-06 — `$500,000` liquid assets, `$1,000,000` net worth
- **Total test coverage:** 12 common + 9 Arby's-specific = **21 tests from 1 class**

3c. `HomePageTest.java` (5 min)

Open: `src/test/java/com/inspire/tests/HomePageTest.java`

Talking points:

- Tests the home page `/` independently — brand-agnostic
- TC-H-01: URL check, TC-H-02/03: hero heading, TC-H-05: GET STARTED navigation
- Point to `homePage.isBrandInDropdown(Brand.ARBYS.getDisplayName())` — TC-H-07
- Every test calls `logStep()` and `logPass()` — these auto-capture screenshots (show a screenshot in the report later)

PART 4 — Configuration & Test Data (5 min)

4a. `config.properties`

Open: `src/test/resources/config.properties`

Talking points:

- Controls browser, headless mode, base URL, active brand, timeouts
- Show system property override priority: `-Dheadless=true` beats the file
- Demo commands:

```
.\mvnw.cmd clean test -P arbys -Dbrowser=firefox -Dheadless=true
```

4b. `brands/arbys.properties`

Open: `src/test/resources/brands/arbys.properties`

Talking points:

- Brand-specific data overlay — loaded on top of `config.properties` when `active.brand=arbys`
 - Shows `brand.liquid.assets=500,000`, `brand.net.worth=1,000,000`
 - Show the pattern: every brand gets its own properties file
 - New brand = new properties file, zero framework changes
-

PART 5 — Page Objects & Driver (15 min)

5a. `ArbysPage.java` (4 min)

Open: `src/main/java/com/inspire/pages/brands/ArbysPage.java`

Talking points:

- `extends AbstractBrandPage` — Liskov Substitution in action
- `@FindBy` annotations — Page Factory pattern, no `driver.findElement()` in test code
- `getBrandPagePath()` returns `"/arbys"` — only brand-specific thing
- Point out apostrophe handling in XPath: `contains(., 'Why')` and `contains(., 'Arby')` — not `"Why Arby's"`
- `isLiquidAssetsRequirementDisplayed()` → ties back to TC-A-05

5b. `AbstractBrandPage.java` (4 min)

Open: `src/main/java/com/inspire/pages/brands/AbstractBrandPage.java`

Talking points:

- Common `@FindBy` locators shared by ALL brands — `helpSectionHeading`, `whatNextHeading`, `stepApply` through `stepSign`
- `implements IBrandPage` — Dependency Inversion, tests depend on the interface
- `clickGetStartedAndGetUrl()` — show the `waitForUrlContains()` + href fallback (explain P-03 pitfall)
- `isGetStartedButtonDisplayed()` — scrolls before checking visibility (explain P-02 pitfall)

5c. `HomePage.java` (3 min)

Open: `src/main/java/com/inspire/pages/HomePage.java`

Talking points:

- `implements INavigable` — Interface Segregation: navigation methods separate from brand-page methods
- `hoverOver(ourBrandsMenuTrigger)` — JavaScript hover for Squarespace dropdown
- `brandsSectionHeading` locator — show the split `contains()` for "Inspire's"

5d. `DriverManager.java` (4 min)

Open: `src/main/java/com/inspire/driver/DriverManager.java`

Talking points:

- `ThreadLocal<WebDriver>` — one **WebDriver per thread** for safe parallel execution
- `initDriver()` / `getDriver()` / `quitDriver()` — clear lifecycle
- Chrome options: `--headless=new`, `--no-sandbox`, `--window-size=1920,1080`

- `WebDriverManager.chromedriver().setup()` — no manual ChromeDriver download needed
- Called from `BaseTest.setUpMethod()` / `tearDownMethod()` — developers never call this directly

5e. `WaitUtils.java` (bonus 3 min)

Open: `src/main/java/com/inspire/utils/WaitUtils.java`

Talking points:

- `waitForVisibility()`, `waitForClickability()`, `scrollIntoView()` — explicit waits only, implicit wait = 0
- `scrollIntoView()` is the key fix for Squarespace scroll animations
- `waitForUrlContains()` — used in `clickGetStartedAndGetUrl()` fallback

PART 6 — TestNG Suite XML Files (5 min)

Open both: `testng-suites/testng-arbys.xml` and `testng-suites/testng-all.xml`

Talking points for `testng-arbys.xml`:

- Two `<test>` nodes: Smoke Tests + Full Regression — run in order
- `<groups><run><include name="smoke"/>` — selective execution
- Maven profile `-P arbys` points Surefire at this file

Talking points for `testng-all.xml`:

- Shows the extensible structure: add a `<test>` block for each new brand
- Future brand stubs are commented in — one uncomment to enable

Show the Maven profiles in `pom.xml`:

- `-P arbys` → runs `testng-arbys.xml`
- `-P all-brands` → runs `testng-all.xml`
- `-Dgroups=smoke` → runs only smoke group across whichever suite

PART 7 — Live Test Run + Report Demo (10 min)

Run in terminal:

```
.\mvnw.cmd clean test -P arbys -Dgroups=smoke
```

While tests run, narrate:

- Maven downloads dependencies on first run (show the download progress)
- `WebDriverManager` silently downloads `ChromeDriver`

- Chrome browser opens, cookie banner dismissed automatically
- Tests execute — point out the browser navigating the Arby's page

After run completes, open the report:

```
test-output\extent-reports\<timestamp>\report.html
```

Walk through the report:

- Dashboard: pass/fail count, timeline bar
- Click TC-B-03 → show step-by-step log with inline screenshots
- Click a failed test (if any) → show "Screenshot at failure" screenshot
 - *"This tells you the exact browser state when it failed"*
- System/Environment panel: Browser, Active Brand, Java version, OS

PART 8 — GitHub Copilot Custom Agent (20 min)

8a. Tools & Their Permissions (5 min)

Open: `.github/agents/Inspire-Franchise-Selenium-Automation-Framework-Agent.agent.md`

Show the YAML frontmatter:

```
tools: [vscode/getProjectSetupInfo, execute/runTests, execute/testFailure,
        read/readFile, edit/createFile, search/codebase, web/fetch, ...]
```

Group them for the audience:

Category	Tools	What they enable
<code>vscode/*</code>	installExtension, memory, runCommand	IDE integration, remembers session state
<code>execute/*</code>	runInTerminal, runTests, testFailure	Run Maven, read test failures
<code>read/*</code>	readFile, problems	Read any Java/XML/properties file
<code>edit/*</code>	createFile, editFiles	Create DunkinPage.java, update factory
<code>search/*</code>	codebase, textSearch, usages	Navigate the entire codebase
<code>web/*</code>	fetch, openBrowserPage	Fetch live site DOM for new locators

Key point: *"The agent can open the actual franchising website, read the DOM, and write the correct `@FindBy` locator for a new brand — without you having to inspect the page yourself."*

8b. MANDATORY Rules — Why They Matter (5 min)

Still in the agent file, show the three MANDATORY sections:

MANDATORY — Always Use `clean test` :

"Every time a developer types `mvnw test` without `clean` and hits `Cannot instantiate class`, they lose 20 minutes. This rule costs zero tokens and saves hours."

MANDATORY — Cookie Banner:

"Maven Surefire starts a fresh Chrome with no cookies. The banner appears every single time. Without `dismissCookieBannerIfPresent()`, every test fails at the click step."

MANDATORY — XPath Rules:

- Open `ArbysPage.java`, show `whyArbysHeading` locator

"The apostrophe in 'Arby's' is a Unicode curly quote (U+2019). Ask Copilot without this rule — it writes `"Why Arby's"` — instant `SyntaxError`. With the rule — it always splits into two `contains()` calls."

8c. Project Structure in the Agent (3 min)

Show the Project Structure section in the agent file:

"The agent knows every file's responsibility. When you say 'add a Sonic brand', it creates `SonicPage.java`, adds `case SONIC:` to the factory, creates `SonicTest.java`, and creates `testng-sonic.xml` — in the right packages, with the right class signatures."

"Without this section, Copilot guesses the package structure. With it, the first answer is correct."

8d. Common Pitfall Demo — GET STARTED Button (7 min)

The most impactful demo — live with the agent selected in Copilot chat.

Switch to Copilot chat, select the agent, then ask:

"The GET STARTED button on the Arby's page is not clicking correctly. What could be wrong?"

Agent's answer should reference P-03:

- Multiple hidden nav copies of the button exist
- Locator needs `not(ancestor::header)`, `not(ancestor::nav)`, `contains(@href, 'franchise-with-us')`
- `clickGetStartedAndGetUrl()` already has the `waitForUrlContains()` + href fallback

Then ask:

"Add Baskin Robbins brand to this framework"

Watch the agent:

1. Create `BaskinRobbinsPage.java` in the correct package
2. Add `case BASKIN_ROBBINS:` to `BrandPageFactory.java`
3. Create `BaskinRobbinsTest.java` extending `AbstractBrandTest`
4. Offer to create `testng-baskin-robbins.xml`

Close with:

"The 12 common tests from `AbstractBrandTest` run automatically for the new brand. The agent saved roughly 4 hours of boilerplate and zero existing files were changed — that's the Open/Closed Principle enforced by the agent, not just documented."

Q&A Prompts (to fill time if needed)

- "What would break if we used `mvn test` instead of `.\mvnw.cmd clean test`?"
- "Why is there a `ThreadLocal` in `DriverManager`? What problem does it solve?"
- "The `@FindBy` annotation uses `normalize-space(.)` — what's the dot?"
- "How do we add a CI pipeline trigger for just the smoke group?"
- "What's the difference between Ask, Plan and Agent mode in Copilot?"

Pre-Session Checklist

- ☐ Browser open at <https://www.franchising.inspirebrands.com/>
- ☐ VS Code open at `C:\code\Inspire\InspireFranchises`
- ☐ Terminal ready in project root
- ☐ Copilot agent pre-selected: `Inspire-Franchise-Selenium-Automation-Framework-Agent`
- ☐ Test run done once beforehand (Maven dependencies already downloaded — live run will be fast)
- ☐ Previous Extent report open as reference in browser
- ☐ Presentation slides open: `team-presentation\GitHub-Copilot-for-Automation-Frameworks.pptx`